Pontifícia Universidade Católica de Minas Gerais

Curso de Engenharia de Software

Disciplina: Teste de Softare

Luiz Felipe Vieira

Matrícula 754921

Belo Horizonte

10/10/2026

1. Análise dos smells

A suíte original foi preservada em `test/userService.smelly.test.js`. Foram identificados os seguintes problemas:

1.1 Teste Eager e falta de foco

O teste deve criar e buscar um usuário corretamente executa duas ações de negócio diferentes: cria um usuário e depois o busca. Quando ele falha, n**ão fica imediatamente claro se o problema está na criação ou na consulta.** Na suíte limpa, essas responsabilidades foram separadas em testes independentes, cada um com um único objetivo.

O teste de desativação original também valida dentro de um único caso o comportamento de usuário comum e de administrador. A refatoração criou um teste para cada regra de negócio.

1.2 Lógica condicional no teste

O teste de desativação percorre uma lista com *for* e decide o que verificar usando *if*. Isso torna o teste mais difícil de ler e faz com que as **asserções dependam do caminho de execução**. Uma alteração no conjunto de dados pode reduzir a cobertura real sem deixar o propósito evidente.

Na suíte limpa, os cenários foram escritos separadamente: um teste verifica a desativação de usuário comum, outro verifica que administradores permanecem ativos e um terceiro verifica identificadores inexistentes.

1.3 Exceção capturada manualmente

O teste de menor de idade usa *try/catch* e **só executa expect dentro do catch. Se createUser deixar de lançar a exceção, o teste termina sem asserção e pode passar silenciosamente.** Esse é um teste frágil porque não garante o comportamento que pretende proteger.

A refatoração usa expect(() ⇒ ...).toThrow(...), que falha explicitamente quando a exceção não é lançada.

1.4 Teste desabilitado

O caso de relatório vazio está marcado com test.skip e não possui asserções. Portanto, ele **não oferece proteção contra regressões**. Na suíte limpa, o cenário foi implementado e verifica a mensagem retornada quando não existem usuários.

2. Processo de refatoração

Antes:

```
test('deve falhar ao criar usuário menor de idade', () => {
  try {
    userService.createUser('Menor', 'menor@email.com', 17);
  } catch (e) {
    expect(e.message).toBe('O usuário deve ser maior de idade.');
  }
});
```

Depois:

```
test('rejeita a criação de usuário menor de idade', () => {
  const createUser = () => userService.createUser('Menor', 'menor@email.com', 17);

  expect(createUser).toThrow('O usuário deve ser maior de idade.');
});
```

O código refatorado deixa o Arrange explícito na criação da função, o Act encapsulado na chamada de *createUser* e o Assert garantido por `toThrow`. A asserção não depende de um caminho condicional, portanto o teste falha corretamente se a validação for removida.

Na suíte completa, os testes foram separados por comportamento, receberam nomes descritivos e passaram a usar matchers de comportamento, como *`toMatchObject`*, *`toBeNull`*, *`toContain`* e *`toThrow`*. IDs aleatórios são verificados pelo tipo e pela existência, em vez de comparar valores gerados pela implementação.

3. Configuração e relatório do ESLint

Foi instalado o ESLint 8.57.1 com eslint-plugin-jest. A versão foi fixada porque o enunciado usa .eslintrc.json, formato suportado pelo ESLint 8.

A primeira análise da suíte original foi:

```
/workspaces/test-smelly/test/userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

6 problems (4 errors, 2 warnings)
```

A ferramenta automatizou a identificação de dois smells visíveis: as asserções condicionais e o teste desabilitado sem asserção. A análise manual complementa o lint ao identificar também o teste Eager e a fragilidade do `try/catch`.

4. Validação final

Comandos executados:

```
npx eslint test/userService.clean.test.js
```

Resultado: nenhum erro ou aviso na suíte limpa.

```
npm test -- --runInBand
```

Resultado: 2 suítes passaram; 13 testes ativos passaram e 1 teste original permaneceu pulado, pois o arquivo original não deveria ser modificado.

O lint global continua reportando somente os problemas intencionais de userService.smelly.test.js. O arquivo `userService.clean.test.js` permanece sem problemas.

5. Conclusão

Testes limpos tornam a causa de uma falha mais fácil de localizar e reduzem a chance de regressões passarem despercebidas. A separação dos cenários, a remoção de lógica condicional e o uso de asserções próprias do Jest tornam a suíte mais legível e confiável. O ESLint complementa a revisão manual ao apontar padrões de risco automaticamente, funcionando como uma verificação contínua de qualidade durante a manutenção do projeto.